

ASSIGNMENT - 3

(23CON1068)

PAGE NO.:

• Section - A :

(1) What is race condition?

Ans. A race condition is an undesirable situation that occurs when a device or system attempts to perform two or more operations at the same time.

(2) Why lock variables are used?

Ans. Lock variables are used to synchronize access to shared resources in concurrent programming. They ensure that only one thread or process can access a shared resource at a time, preventing data races and other concurrency errors. Here are some specific reasons why lock variables are used:

- To prevent race conditions.
- To protect critical sections.
- To implement synchronization primitives.

Q3. What is critical region?

Ans. A critical region is a section of code that accesses shared data and needs to be protected from concurrent access to prevent race conditions and other concurrency errors. Critical regions are typically protected using locks or other synchronization primitives.

Q4. What is Blocking send in Message passing model?

Ans. In a message passing model, a blocking send is a send operation that blocks the sender until the message has been received by the receiver. This means that the sender cannot continue executing until the receiver has acknowledged that it has received the message. Blocking sends are typically used in

Synchronous communication, where the sender needs to know that the receiver has received the message before it can continue executing.

Q5. What is the use of claim edge in Resource allocation graph?

Ans. A claim edge in a Resource allocation graph (RAG) is a dotted line that points from a process to a resource. It indicates that the process may request the resource in the future. Claim edges are used to detect and prevent deadlocks. Claim edges are an important tool for preventing deadlocks in resource allocation systems. By ensuring that a process can only request a resource if it already has a claim edge to the resource.

• Section-B:

Q1. What is Semaphore? Explain it.

Ans. A Semaphore is a synchronization primitive that allows multiple threads or processes to access a shared resource in a controlled manner. Semaphores are typically used to implement mutual exclusion, which is a mechanism to ensure that only one thread or process can access a shared resource at a time.

Semaphores work by using a counter. The counter represents the number of available permits to access the shared resource. When a thread or process wants to access the shared resource, it must first acquire a permit. If it can access the shared resource. When the thread or process is finished with the shared resources, it must release the permit back to the semaphore.

Semaphores can be used to implement a variety of synchronization patterns, such as:

- Mutual exclusion: Semaphores can be used to implement mutual exclusion, which is a mechanism to ensure that only one thread or process can access a shared resource at a time.
- Bounded Buffer: Semaphores can be used to implement a bounded buffer, which is a data structure that can store a limited number of elements.
- Barrier: Semaphores can be used to implement a barrier, which is a synchronization point that all threads or processes must reach before continuing.

Semaphores are two types: —

- (1) Binary Semaphore: This is also known as a mutex lock. It can have only two values - 0 and 1. Its value is initialized to 1. It is used to implement the solution of critical section problem with multiple processes.
- (2) Counting Semaphore: Its value can range over an unrestricted domain. It is used to control access to a resource that has multiple instances.

Now let us see how it does so.

```
P(Semaphore s) {
    while (S == 0);
    S = S - 1;
}
```

```
V(Semaphore s) {
    S = S + 1;
}
```

```
Process P
// code code
P(s);
// critical section
V(s);
// remainder section
```


Q2. What is the difference betⁿ: Deadlock Prevention and Avoidance?

Ans: Deadlock Prevention and Deadlock avoidance are two techniques used to prevent deadlocks in OS and other concurrent systems.

Deadlock Prevention work by ensuring that at least one of the four conditions necessary for a deadlock to occur is never met. these four conditions are:

- (i) Mutual exclusion:- each resource is held by at most one process at a time.
- (ii) Hold and wait:- at least one process is holding a resource and waiting for another resource.
- (iii) NO Preemption:- resources can't be forcibly taken away from a process.
- (iv) Circular wait:- There is a chain of two or more processes, each of which is waiting for the resource held by the next process in the chain.

- Deadlock Prevention can be implemented using a variety of techniques, such as:
- Resource ordering: resource are ordered in a hierarchy and processes are only allowed to request resources in order ascending order.
- Deadlock detection:- the system periodically check for deadlocks, and if one is found, it terminates one of the processes involved in the deadlocks.
- Resource allocation:- the system carefully allocate resources to processes in order to avoid deadlocks.

- **Deadlock Prevention**:- A deadlock prevention method assures that at least one of the four deadlock conditions never occurs. In general, deadlock prevention is impossible to implement, and knowing the total number of current resources, availability and request is not needed to avoid a deadlock situation.
- **Deadlock Avoidance**:- A system is considered safe when all processes may be assigned resources in any order without causing a deadlock. The deadlock avoidance mechanism prevents the system from coming to an unsafe state. The system should be aware of the number of available resources, availability and requests to prevent the system from entering an unsafe condition.

→ difference:-

Characteristic	Deadlock Prevention	Deadlock avoidance
Goal	Prevent deadlocks from occurring.	Avoid deadlocks from occurring.
Method	Ensure that at least one of the four conditions necessary for a deadlock to occur is <u>never</u> met.	Dynamically detect and avoid situations that could lead to deadlocks.
Requirements	Knowledge of the system's resources and the resource requirements of all processes.	Knowledge of the system's resources, the resource requests of all processes and the maximum resource requirements of all processes.
Complexity	Simple to implement.	More complex to implement.
Performance	Lower system utilization.	Higher system utilization.

Feature	Deadlock Prevention	Deadlock Avoidance
Definition	It assures that at least one of the four deadlock conditions never occurs.	It prevents the system from coming to an unsafe state.
procedure	It prevents deadlock by limiting the resource request process and resource handling.	It automatically evaluates requests and determines whether they are safe for the system.
resource Allocation strategy.	It is conservative in deadlock prevention.	It is not conservative in deadlock avoidance.
Preemption Algorithm	It occurs more frequently. Some algorithms are used in deadlock prevention, including non-blocking synchronization and serializing tokens.	There is no preemption in D.A. The most widely used algorithm in deadlock avoidance is Banker's algorithm.

Q3.

Ans.

Explain the precedence Graph with an example?

Precedence graph is a directed acyclic graph which is used to show the execution level of several processes in operating system. It consists of nodes and edges. Nodes represent the processes and the edges represent the flow of execution.

A precedence graph is acyclic meaning that there are no cycles in the graph. This means there is no sequence of tasks that can be executed in such a way that a task depends on itself.

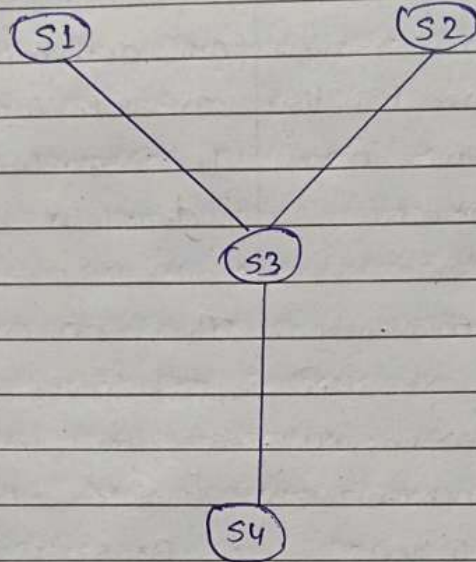
Dec 22, 2023, 17:32 Consider the following code:

S1 : $a = x + y ;$

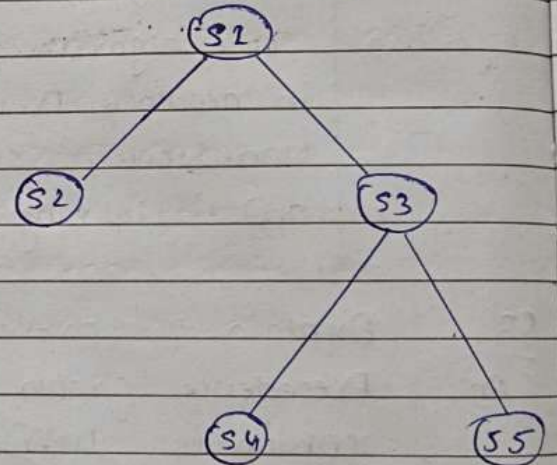
S2 : $b = z + 1 ;$

S3 : $c = a - b ;$

S4 : $d = c + 1 ;$



- S1 per begin
- S2, S3 per begin
- S4, S5 per end.



A Precedence graph can be used to detect and prevent deadlocks in concurrent systems. A deadlock occurs when two or more processes are each waiting for a resource that is held by the other process. By analyzing the Precedence graph, it is possible to detect situations that could lead to deadlocks and take steps to avoid them.

Precedence graph are a Powerful tool for modeling and analyzing concurrent systems. By using precedence graphs, we can detect and prevent deadlocks, schedule tasks efficiently and optimize resource utilization.

• Section - C :

Q1. What is Producer Consumer Problem? Give the solution with sleep and wakeup approach.

Ans. The Producer-consumer Problem is a classic concurrency problem in computer science. It describes two processes, a producer and a consumer, that share a common, bounded buffer. The producer produces items and places them in the buffer, while the consumer removes items from the buffer and consumes them. The problem is to design a solution that allows the producer and consumer to operate concurrently without interfering with each other.

• Solution using sleep and wakeup :

One way to solve the producer-consumer problem is to use the sleep and wakeup system calls. The producer checks to see if the buffer is full before producing an item. If the buffer is full, the producer puts itself to sleep until the consumer wakes it up. Once the producer has produced an item, it wakes up the consumer.

The consumer checks to see if the buffer is empty before consuming an item. If the buffer is empty, the consumer puts itself to sleep until the producer wakes it up. Once the consumer has consumed an item, it wakes up the producer.

One common solution to the producer-consumer problem is the "sleep and wakeup" approach using semaphores or other synchronization mechanisms.

Semaphores are integer variables used for controlling

access to shared resources. In this approach, we typically use three semaphores:

- (1) Empty: Tracks the no. of empty slots in the buffer.
- (2) Full: Tracks the no. of filled slots in the buffer.
- (3) Mutex (or binary semaphore):- Ensures mutual exclusion for accessing the buffer.

• Here are general outline of the soln. using sleep and wakeup:-

- (1) Initialize the semaphores (Empty, Full and mutex) and set their initial values accordingly.

- (2) The Producer Process:

- wait on the Empty semaphore to ensure there's an empty slot in the buffer.
- wait on the Mutex semaphore to gain access to the buffer.
- produce an item and add it to the buffer.
- signal the Mutex semaphore to release the buffer for other processes.

- (3) The Consumer Process:

- wait on the full semaphore to ensure there's a filled slot in the buffer.
- wait on the mutex semaphore to gain access to the buffer (mutual exclusion).
- consume an item from the buffer.
- signal the mutex semaphore to release the buffer for other processes.
- signal the Empty semaphore to indicate that an empty slot is available in the buffer.

The following pseudocode shows a solution to the Producer - Consumer Problem using sleep

and wakeup:

##

// Producer

while (true) {

item = produce - item();

if (buffer - is - Full()) {

// Put the producer to sleep

sleep();

}

put - item - in - buffer(item);

// Wake up the consumer

wakeup();

}

// consumer

while (true) {

if (buffer - is - empty()) {

sleep();

}

// Remove an item from the buffer

item = remove - item - from - buffer();

consume - item(item);

// Wake up the producer

wakeup();

}

• Advantages of sleep and wakeup approach:

→ The sleep and wakeup approach is simple to implement and understand.

→ It is efficient, as it avoids busy waiting.

→ It is flexible and can be used to implement different variations of the producer-consumer problem.

- Disadvantages:
- it can lead to performance overhead.
- it can be difficult to debug.

Q2. What are Deadlock detection methods for single instance and multiple instances of resources.

Ans. Deadlock detection methods are used to identify and resolve deadlock situations in a computer system. Deadlocks occur when two or more processes are unable to proceed because they are each waiting for a resource that another process holds. Deadlock detection is crucial for maintaining system reliability and preventing processes from becoming permanently blocked. There are different methods for deadlock detection, and they can be applied to scenarios with both single and multiple instances of resources.

- For Single Instance Resources:

(1) Wait-Die and Wound-Wait Schemes:

- Wait Die :- Processes requesting a resource may be allowed to wait or may be aborted if their priority is lower than the process holding the resource.
- Wound-Wait :- Processes requesting a resource may be forced to wait or may preempt the resource holder if their priority is lower.

(2) Resource Allocation Graph (RAG):

- in a single-instance resource system, a directed graph can be used to represent resource allocation and process request relationships.
- Detecting a cycle in the RAG indicates the presence of a deadlock.

(3) Timeout Mechanisms: if a process waits for a resource for an extended period, a timeout mechanism can be used to abort the waiting process.

- For Multiple Instance of Resources:

(1) Resource Allocation Graph (RAG):-

- The RAG can also be used for systems with M.I.R.
- Detecting a cycle in the RAG indicates a possible deadlock.
- To find the exact processes involved in the deadlock, further analysis is required.

(2) Banker's Algorithm:-

- The Banker's Algorithm is a resource allocation and deadlock avoidance method.
- It maintains a record of the available resources and checks if a process's request can be granted without causing a deadlock.

(3) Wait-for-Graph:

- A wait-for-graph is used to represent the waiting relationship betⁿ processes.
- It includes processes waiting for resources and processes holding resources.
- A cycle in the wait-for-graph indicates a deadlock.

(4) Resource Matrix Approach:

- A matrix can be used to represent the resource allocation status, request status, and the number of available resources.
- Analyzing the matrix can help detect and resolve deadlocks.

Deadlock detection methods can vary in their complexity and efficiency.

Q3. Write banker's algorithm and give the solution for given state -

Process	Maximum				Allocated				Needed				Available			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
0	6	0	1	2	4	0	0	1	2	0	1	1	3	2	1	1
1	1	7	5	0	1	1	0	0	0	6	5	0				
2	2	3	5	6	1	2	5	4	1	1	0	2				
3	1	6	5	3	0	6	3	3	1	0	2	0				
4	1	6	5	6	0	2	1	2	1	4	4	4				

Solⁿ.

The Banker's algorithm is a resource allocation and deadlock avoidance algorithm. It works by maintaining a record of the available resources and the maximum and current resource needs of each process. The algorithm then checks if a process's request can be granted without causing a deadlock. If a request cannot be granted, the system considers it unsafe and does not allocate the resources.

→ Steps in the Banker's algorithm -

(1) Initialize : initialize the following variables

- Available resources
- Maximum resources needs
- Current resources needs

(2) Check for safety :- Iterate over each process in the system.

(3) Allocate resources : If the system is in a safe state, then allocate the requested resources to the process and update the current resource needs of the process and the available resource.

(4) Repeat : Repeat steps 2 and 3 until all processes have

• Applying Banker's Algorithm.

Available Resource of A, B, C and D are 3, 2, 1 and 1.

Now we check if each type of resource request is available for each process.

Step-1; for P₀

Need $\leq$ Available

$$2, 0, 1, 1 \leq 3, 2, 1, 1$$

Condition is true

New available = Available + Allocation

$$= 3, 2, 1, 1 + 4, 0, 0, 1$$

$$= 7, 2, 1, 2$$

Step-2; for P₁

Need $\leq$ Available

$$0, 6, 5, 0 \leq 7, 2, 1, 2$$

Condition is false

Step-3; for P₂

Need $\leq$ Available

$$1, 1, 0, 2 \leq 7, 2, 1, 2$$

Condition is true

New available = $7, 2, 1, 2 + 1, 2, 5, 4$

$$= 8, 4, 6, 6$$

Step-4; for P₃

Need $\leq$ Available

$$1, 0, 2, 0 \leq 8, 4, 6, 6$$

Condition is true

New available = $8, 4, 6, 6 + 0, 6, 3, 3$

$$= 8, 10, 9, 9$$

Step-5;for P_4 Need $\leq$ Available $1, 4, 4, 4 \leq 8, 10, 9, 0$

Condition is true

$$\text{New available} = 0, 2, 1, 2 + 8, 10, 9, 0$$

$$= 8, 12, 10, 11$$
Now, we again examine for P_1 Step-6;for P_1 Need $\leq$ Available $0, 6, 5, 0 \leq 8, 12, 10, 11$

Condition is true

$$\text{New available} = 1, 1, 0, 0 + 8, 12, 10, 11$$

$$= 9, 13, 10, 11$$

Here we execute ~~benker's~~ algorithm to find
safe state and safe sequence like

 P_0, P_2, P_3, P_4, P_1